Day 10 • Recursion basics

1️⃣ Recursion kya hota hai?

👉 Jab koi function khud ko hi call karta hai, use recursion kehte hain.

Example (Simple)

```
void printHello(){
    System.out.println("Hello");
    printHello(); // function khud ko call kar raha hai
}
```

⚠️ Ye infinite loop ban jayega (kyunki stop condition nahi hai)

2️⃣ Important Concept

Recursion me 2 cheezein zaroor hoti hain:

✅ 1. Base Case (Stopping condition)

👉 Jab function rukta hai

✅ 2. Recursive Call

👉 Function khud ko dobara call karta hai

3️⃣ Example – Print 1 to N

```
package Day_10;

public class RecursionBasic {
    //jab hum same function ko bar bar call krte hai to recursion bolte hai
    //Example
    public static void printNum(int n){
        if(n==0){
            return;
        }
        printNum(n-1);

        System.out.println(n);
    }
    public static void main(String[] args) {
        printNum(5);
    }

}
```

🔥 Dry Run

Call stack:

printNum(5)
printNum(4)
printNum(3)
printNum(2)
printNum(1)
printNum(0) → stop

Output:

1 2 3 4 5

4️⃣ Example – Factorial

Formula:

n! = n × (n-1)!

Java Code

```
public class Factorial {
    public static int fact(int n){

        if(n == 0 || n == 1){
            return 1; // Base case
        }

        return n * fact(n-1); // Recursive call
    }

    public static void main(String[] args) {
        System.out.println(fact(5));
    }
}
```

🔥 Dry Run (fact(5))

5 * fact(4)
5 * 4 * fact(3)
5 * 4 * 3 * fact(2)
5 * 4 * 3 * 2 * fact(1)
5 * 4 * 3 * 2 * 1
= 120

5️⃣ Example – Fibonacci

Series:

0 1 1 2 3 5 8

Java Code

```
public class Fibonacci {
    public static int fib(int n){

        if(n == 0) return 0;
        if(n == 1) return 1;

        return fib(n-1) + fib(n-2);
    }

    public static void main(String[] args) {
        System.out.println(fib(6));
    }
}
```

Output:

8

6️⃣ Call Stack Concept (Very Important)

👉 Recursion me stack use hota hai

Har function call stack me jata hai

Jab base case milta hai → stack unwind hota hai

7️⃣ Time Complexity

Problem	Complexity
Print N	O(n)
Factorial	O(n)
Fibonacci	O(2^n) ❌ (slow)